Assignment 1

CSA0903-Java Programming

Name: Sowmiya. D

Reg No.:192511063

Scenario — Campus Smart-Parking System

The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors, and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles (EVs). EVs may also use charging slots, which are charged separately.

When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number, and record the entry time. When the vehicle leaves, the system must release the slot and compute the parking fee. Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.

The facility may contain:

1 floor × 20 slots → 3 floors × 50 slots → 5 floors × 100 slots

You are required to analyse the problem, design an object-oriented model for it, implement the model in Java and validate it. Your design must remain correct and easy to extend as the facility and its rules grow.

Design Challenge:

The university now wants to add new vehicle categories and a new pricing rule without rewriting the classes that already work.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 3 – Design the Control Logic 4 Marks

Bloom's Level: K4 – Analyse

Design the decision and looping logic for the two core operations: **allocating a free slot** and **computing the parking fee**.

Discuss:

- How the program searches the floors and slots for a suitable free slot, and when the search stops
- How the fee slabs (for example: first hour, each additional hour, daily maximum) are expressed using if–else or switch
- How break, continue and return are used to avoid unnecessary work
- What happens when no suitable slot is free

Write **pseudocode for both operations** following the SIMATS Engineering – Standard for Algorithmic Pseudocode before writing Java code.

Complete the Code

Complete the class below and execute it. Do not change the main() method.

```
public class ParkingFacility {
   // slots[floor][slot] holds a vehicle number, or null if the slot is free     private final String[][] slots;

   public ParkingFacility(int[] slotsPerFloor) {         slots = new
String[slotsPerFloor.length][];         for (int f = 0; f < slotsPerFloor.length; f++) {
slots[f] = new String[slotsPerFloor[f]];
      }
   }

   /** Returns {floor, slot} of the first free slot, or null if it is full. */     public int[] allocate(String vehicleNo) {
      // TODO 1: Search floor by floor and slot by slot; place vehicleNo in the first
      //   free slot and return {floor, slot}


      return null;     }

   /** Frees a slot. Returns false if the position is invalid or already free. */     public boolean release(int floor, int slot) {
      // TODO 2: Validate floor and slot, free the slot and return true; otherwise
      //   return false


   }

   public static void main(String[] args) {
      // floor 0 has 2 slots, floor 1 has 1 slot
      ParkingFacility p = new ParkingFacility(new int[]{2, 1});
      System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
      System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));
      System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));
      System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
      System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " "
            + p.release(5, 0));
      System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
   }
}
```

Expected output:

```
[0, 0]
[0, 1] [1, 0]
null
true false false
[0, 1]
```

Alternative Concept and Complexity Analysis

Your allocation scans every floor and slot until a free one is found. If, instead, you maintained a count of free slots per floor and a stack of free slot numbers, what would the allocation and release logic become? Analyse the time complexity of allocate() and release() and the extra space needed in both approaches for F floors and S slots per floor.

Evidence Expected

Complete pseudocode for slot allocation and fee computation with correct decision and loop logic.

Solution:

TASK 03 – Design the Control Logic

1. Slot Allocation Logic

Slot allocation is the process of finding an available parking slot for an incoming vehicle.

The system searches each floor and checks every slot. If a free slot is found, the vehicle number is stored in that slot and its floor and slot positions are returned.

If all slots are occupied, the system returns null.

Steps involved:

1. Start searching from the first floor.
2. Check each slot on that floor.
3. If a slot is free, allocate it to the vehicle.
4. Return the floor and slot number.
5. If no free slot is available, return null.

2. Parking Fee Calculation Logic

The parking fee depends on the vehicle type and parking duration.

For example, the following pricing rules are assumed:

Parking duration	**Fee rule**
First hour	Base hourly rate
Every additional started hour	Additional hourly rate
Daily maximum	Maximum ₹200 per day

The system calculates the billable hours and applies the appropriate fee.

For example, if a car's hourly rate is ₹30 and it parks for 130 minutes:

- Billable hours = 3
- Parking fee = 3 × 30
- Total fee = ₹90

4. Use of Control Statements

Statement	**Purpose**
if-else	Checks whether a slot is free and applies fee rules.

for loop	Searches floors and slots.
break	Stops searching after finding a free slot.
continue	Skips occupied slots.
return	Returns the allocated slot or calculated fee.

5. Pseudocode – Slot Allocation

```
ALGORITHM AllocateSlot(vehicleNo)
BEGIN
   FOR each floor f
      FOR each slot s in floor f
         IF slot[f][s] is occupied THEN
            CONTINUE
         END IF
         slot[f][s] = vehicleNo
         RETURN f, s
      END FOR
   END FOR
   RETURN NULL
END
```

6. Pseudocode – Parking Fee Calculation

Assumption: Every started hour is charged in full, with a daily maximum of ₹200.

```
ALGORITHM CalculateFee(minutes, ratePerHour)
BEGIN
   IF minutes <= 0 THEN
      RETURN 0
   END IF
   hours = minutes / 60
   IF minutes MOD 60 != 0 THEN
      hours = hours + 1
```

```
    END IF
    fee = hours * ratePerHour
    IF fee > 200 THEN
        fee = 200
    END IF
    RETURN fee
END
```

7. Alternative Concept and Complexity Analysis

Alternative: Using a Free-Slot Stack

Instead of searching every floor and slot, the system can maintain a stack of available slot positions for each floor.

- During allocation, take a free slot from the stack.
- During release, return the slot to the stack.
- Maintain a free-slot count for each floor.

Operation	**Scanning approach**	**Stack approach**
Allocate	O(F × S) worst case	O(1)
Release	O(1) with direct position validation	O(1)
Extra space	O(F × S) for parking data	O(F × S) for stacks in addition to parking data

Here, F is the number of floors and S is the maximum number of slots per floor.

The stack approach provides faster allocation but requires additional memory to maintain free-slot information.

8. Java Implementation – Complete the Given Code

Mandatory execution

What you have to do:

1. Open VS Code.
2. Open your Task03 folder.
3. Create a Java file named ParkingFacility.java.
4. Copy the complete code below.
5. Run the program.

6. Compare your output with the expected output in the question.
7. Take a screenshot and save it as output.png.

Complete Java program

```
public class ParkingFacility {
    // slots[floor][slot] holds a vehicle number,
    // or null if the slot is free
    private final String[][] slots;
    public ParkingFacility(int[] slotsPerFloor) {
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) {
            slots[f] = new String[slotsPerFloor[f]];
        }
    }
    /** Returns {floor, slot} of the first free slot,
     * or null if it is full.
     */
    public int[] allocate(String vehicleNo) {
        // TODO 1 completed
        for (int f = 0; f < slots.length; f++) {
            for (int s = 0; s < slots[f].length; s++) {
                if (slots[f][s] == null) {
                    slots[f][s] = vehicleNo;
                    return new int[]{f, s};
                }
            }
        }
```

```
    return null;
}
/** Frees a slot. Returns false if the position
 * is invalid or already free.
 */
public boolean release(int floor, int slot) {
    // TODO 2 completed
    if (floor < 0 || floor >= slots.length) {
        return false;
    }
    if (slot < 0 || slot >= slots[floor].length) {
        return false;
    }
    if (slots[floor][slot] == null) {
        return false;
    }
    slots[floor][slot] = null;
    return true;
}
public static void main(String[] args) {
    ParkingFacility p = new ParkingFacility(new int[]{2, 1});
    System.out.println(java.util.Arrays.toString(
        p.allocate("TN01AB1111")));
    System.out.println(java.util.Arrays.toString(
        p.allocate("TN02CD2222")));
    System.out.println(java.util.Arrays.toString(
        p.allocate("TN03EF3333")));
    System.out.println(java.util.Arrays.toString(
```

```
        p.allocate("TN04GH4444")));
      System.out.println(p.release(0, 1) + " "
        + p.release(0, 1) + " "
        + p.release(5, 0));

      System.out.println(java.util.Arrays.toString(
        p.allocate("TN04GH4444")));
    }
}
```

Expected Output

[0, 0]

[0, 1]

[1, 0]

null

true false false

[0, 1]

